Oracle Database 19c RAC 과정

진단·성능·종합 운영

인터커넥트, gc 대기, 로그 수집, 점검 스크립트, 그리고 진단 순서

제17장

학습 목표

- HAIP 와 인터커넥트 진단 네 단계를 설명한다
- gc 대기 이벤트 이름을 세 부분으로 읽는다
- 경합의 원인 객체와 블록 종류를 특정한다
- tfactl 로 범위를 좁혀 로그를 모은다
- 계층별 점검 스크립트를 설계하고 결함을 고친다
- 1~16장을 진단 순서로 묶는다

Agenda

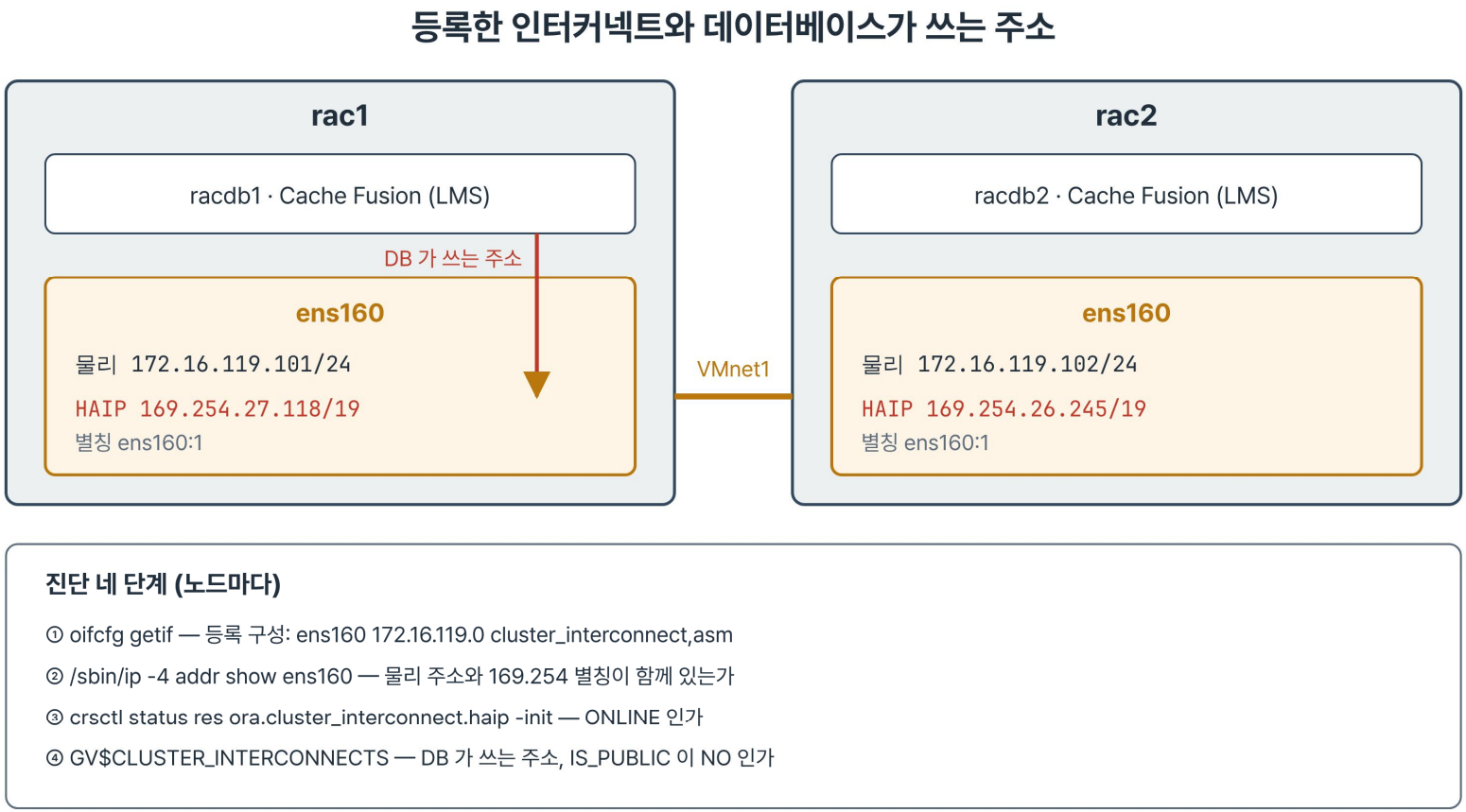
- 17.2 인터커넥트 확인
- 17.3 클러스터 대기 이벤트 / 17.4 원인 객체 특정
- 17.5 로그 수집
- 17.6 종합 점검 스크립트
- 17.7 장애 시나리오 / 17.8 RAC 와 Data Guard
- 17.9 과정을 마치며

17.2

인터커넥트 확인

- DB 가 보고하는 주소
- HAIP 를 쓰는 이유
- 진단 네 단계와 통계

등록한 주소와 DB 가 쓰는 주소



물리 주소 위의 HAIP 별칭

HAIP 를 쓰는 이유

DB 와 물리 네트워크 사이의 이름표

- 인터커넥트를 여러 개 둘 수 있게 한다
- 부하 분산, 한쪽이 죽으면 남은 쪽으로 이동
- DB 는 HAIP 주소만 안다
- ora.cluster_interconnect.haip 는 -init 자원

인터커넥트 진단 네 단계

ping 만으로는 HAIP 장애를 못 찾는다

- ① oifcfg getif — 등록된 인터페이스
- ② /sbin/ip -4 addr show — 169.254 별칭 존재
- ③ crsctl status res ...haip -init — ONLINE
- ④ GV$CLUSTER_INTERCONNECTS — IS_PUBLIC 이 NO

인터커넥트 통계

received 와 served 는 짝을 이룬다

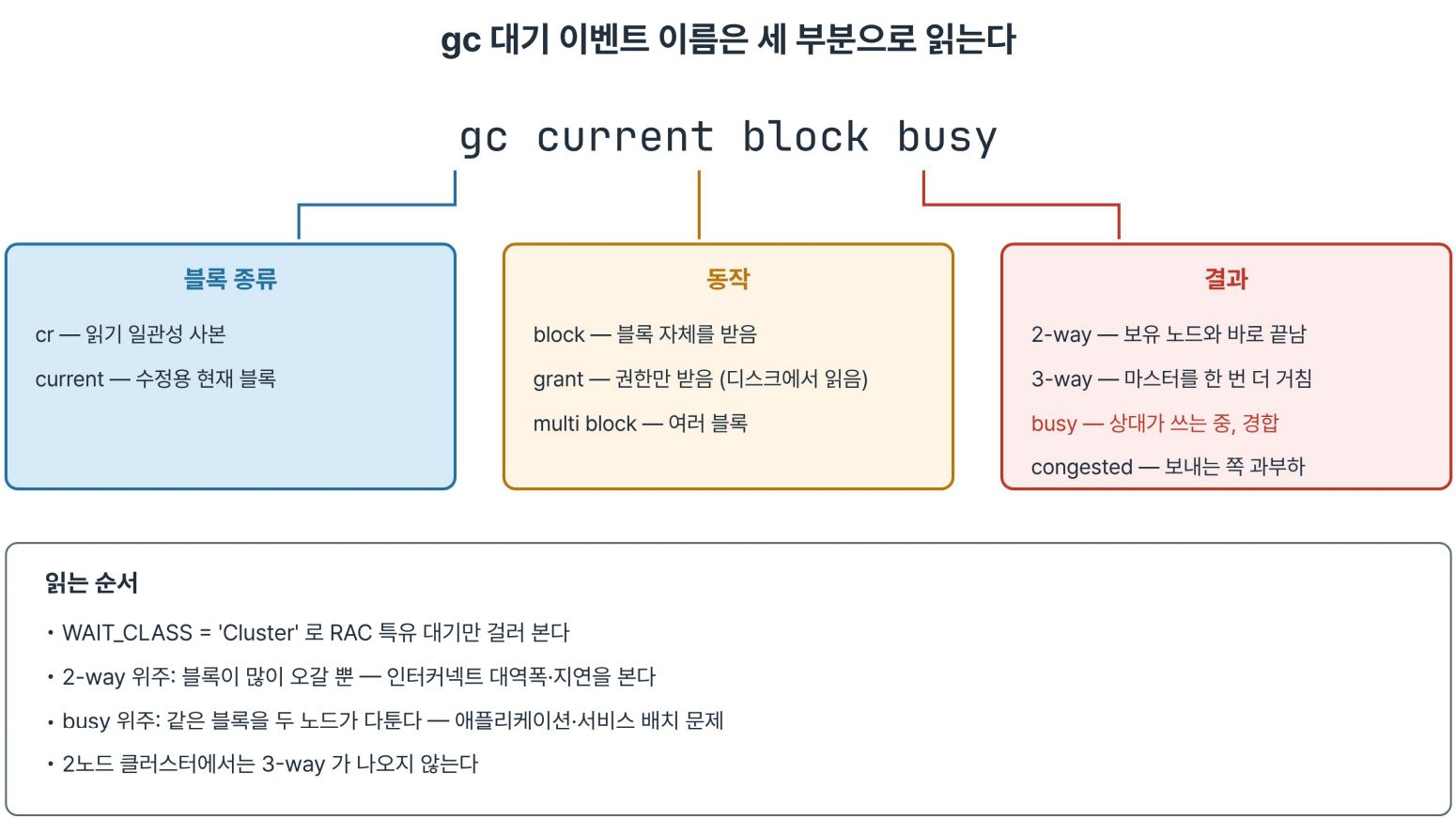
- gc current/cr blocks received · served
- 한쪽이 받은 수 = 다른 쪽이 보낸 수
- gc local grants 가 많을수록 인터커넥트 부담이 적다
- 누적값이므로 전후 증가분으로 비교

17.3

클러스터 대기 이벤트

- 이름 읽기
- 경합을 직접 만든다

gc 대기 이벤트 이름



블록 종류, 동작, 결과

주요 gc 대기

무엇을 먼저 의심하는가

- 2-way: 정상, 양이 많으면 인터커넥트
- grant 2-way: 디스크 읽기, 버퍼 캐시
- block busy · buffer busy: 핫 블록 경합
- congested: 보내는 쪽 LMS 과부하

경합 재현

같은 50 행을 두 노드가 40초 동안 갱신

- ts_ch20 · sys.ch20_t 20000 행
- 양 노드에서 동시에 update id 1~50 반복
- current received +3987 / +3996, served 는 반대로 짝
- 상위 대기: 2-way → gc current block busy

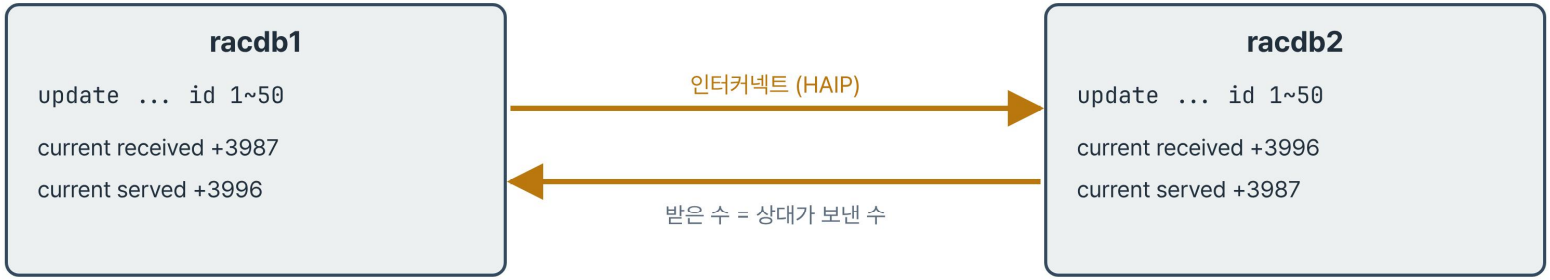
17.4

원인 객체 특정

- GV$SEGMENT_STATISTICS
- GV$INSTANCE_CACHE_TRANSFER
- gc 대기 대응 순서

경합 재현과 원인 특정

gc 경합 재현 — 같은 50 행을 두 노드가 40초 동안 갱신



① 대기 이벤트 순위가 바뀐다

GV$SYSTEM_EVENT

전: gc current block 2-way 1위

후: gc current block busy 1위 (양 노드)

평균 0.19 → 0.42~0.44 센티초

gc buffer busy release 도 증가

무엇이 경합하는지는 아직 모른다

② 원인 객체를 특정한다

GV$SEGMENT_STATISTICS

SYS.CH20_T gc current blocks received

inst 1: 3786 / inst 2: 3787

→ 전체 증가분 거의 전부

GV$INSTANCE_CACHE_TRANSFER

data block CURRENT_BUSY 4407 · 4452

undo header CR_BLOCK 도 크다

이벤트에서 세그먼트로

블록 종류별 전송

undo header 도 오간다

- data block CURRENT_BUSY 4407 · 4452
- undo header CR_BLOCK 이 크다
- CR 사본을 만들려면 상대 UNDO 헤더 필요
- 2노드라 CR_2HOP = CR_BLOCK

gc 대기 대응 순서

서비스 분리가 가장 빠르다

- 전체 DB 시간 대비 비율로 문제인지 판단
- 2-way 위주인가 busy 위주인가
- 세그먼트 특정 — 시퀀스 인덱스 끝, 상태 플래그, 카운터
- 서비스 분리 · 시퀀스 CACHE NOORDER · 설계 수정

17.5

로그 수집

- diagcollection.pl 대신 tfactl
- 범위 지정과 시간 단위

tfactl diagcollect

19c 에는 diagcollection.pl 이 없다

- TFA 데몬이 두 노드에서 실행 (tfactl print status)
- -crs · -database · -asm 로 컴포넌트 지정
- -last 1h, -node local|all, -tag 디렉터리
- -last 5m 은 거부 — 단위는 h, d

언제 무엇을 모으는가

장애 직후, 범위를 좁혀서

- SR 은 -srdc (예: dbrac_evictions)
- 스스로 분석할 땐 컴포넌트와 시간을 좁힌다
- 로그는 회전하므로 장애 직후에
- 수집도 부하 — 급한 불을 끈 뒤에

17.6

종합 점검 스크립트

- 여섯 계층
- FAIL 과 WARN
- 없는 것은 보이지 않는다

여섯 계층

이 순서가 진단 순서다

- 1 클러스터: 스택 6/6, 자원 17, 불일치
- 2 알림 · 3 서비스: ONS, 서비스 배치
- 4 인스턴스: Open 수, SPFILE
- 5 스토리지 · 6 백업: 스레드, UNDO, 아카이브, FRA

FAIL 과 WARN

기준선을 숫자로 박아 둔다

- 인스턴스 미기동 → FAIL, 자원 개수 차이 → WARN
- 종료 코드 = 실패 건수, 스케줄러가 판단
- 기준선 17 · 6 을 고칠 때 = 구성 변경 인지
- 알려진 WARN(asmgroup 3번, chad)은 걸러 낸다

없는 것은 보이지 않는다

있어야 할 것의 목록을 기준으로

- group by thread# → thread 2 행이 통째로 빠짐
- 아카이브를 못 만드는 스레드가 화면에 안 보인다
- V$THREAD 에 아우터 조인 → archived=0
- 인스턴스는 GV$INSTANCE, 자원은 기준선 개수

17.7

장애 시나리오와 RAC · Data Guard

- 계층별 흔적
- 자동 재기동의 한계
- RAC 와 Data Guard

장애 20초 뒤 계층별 흔적



하나의 사건이 여러 계층에

자동 재기동의 한계

CRS-2771 이면 사람이 올린다

- RESTART_ATTEMPTS=2, UPTIME_THRESHOLD=1h
- 한 시간에 두 번 죽으면 재시도 중단
- TARGET 은 ONLINE 그대로
- srvctl start instance 후 반복 장애 원인 조사

RAC 와 Data Guard

서로 다른 장애를 막는다

- RAC: 인스턴스·서버 장애, 데이터 사본 하나
- Data Guard: 스토리지·사이트 장애, 사본 둘 이상
- 논리 오류는 둘 다 못 막는다 → 백업·플래시백
- 준비: ARCHIVELOG · 스레드별 리두 · 공유 아카이브

17.9

과정을 마치며

- 진단 순서로 다시 묶기
- 확인하고 되돌리는 습관

진단 순서

진단은 아래 계층부터 — 1~16장을 한 줄로

① Clusterware	2 · 3 · 11장	`crsctl check cluster -all` · `crsctl stat res -t (-init)` 스택 online, TARGET·STATE 불일치, OCR·보팅 파일
② 노드 멤버십	2 · 3 · 15 · 17장	`olsnodes -n -s -t` · `oifcfg getif` · `HAIP -init` 노드 Active, 인터커넥트·HAIP, 보팅 파일 과반, 인벤토리 노드 목록
③ 디스크그룹	4 ~ 8 · 10장	`asmcmd lsdg` · `V$ASM_DISKGROUP` · `V$ASM_OPERATION` MOUNTED, 여유 공간·Usable, 리밸런스, 오프라인 디스크
④ 인스턴스	1 · 9 · 10 · 14 · 16장	`srvctl status database -v` · `gv$instance` · `v$thread` Open 수, sid.parameter, 스레드·UNDO, 아카이브, 인스턴스 이름
⑤ 서비스 · 리스너	1 · 12 · 13장	`srvctl status service` · `lsnrctl services` · `onsctl ping` 서비스 배치, SCAN·VIP, ORA-12514/12541, FAN·TAF·AC
⑥ SQL	1 · 17장	`gv$system_event` · `gv$segment_statistics` gc 대기(2-way·busy), 원인 세그먼트, 서비스 분리로 경합 해소

Clusterware 에서 SQL 까지

계층별 핵심

각 장을 한 줄로

- Clusterware·멤버십: TARGET/STATE, OCR·보팅, HAIP, 노드 세 곳
- 디스크그룹: 이중화·장애 그룹, 리밸런스, 공유 위치
- 인스턴스: 스레드·UNDO 분리, 인스턴스 복구, 이름
- 서비스·리스너: SCAN·서비스로 접속, FAN·TAF·AC
- SQL: gc busy → 세그먼트 → 서비스 분리

확인하고 되돌린다

성공 메시지보다 실제 상태

- restore spfile to 가 CRS 등록을 바꿨다
- log_archive_dest_1='' 은 비활성화였다
- 인터커넥트 주소는 등록한 것과 달랐다
- addnode.sh 성공에도 인벤토리는 그대로였다
- 모든 실습은 기준선으로 돌아온다

요약

- DB 는 HAIP 별칭으로 인터커넥트를 쓴다. 진단 네 단계
- gc 대기는 종류·동작·결과로 읽는다. busy 는 경합
- 원인은 GV$SEGMENT_STATISTICS 로 특정
- tfactl diagcollect 는 컴포넌트·시간 범위를 지정
- 점검은 계층 순, FAIL/WARN 구분, 있어야 할 것 기준
- CRS-2771 이면 사람이 올리고 원인을 찾는다

퀴즈

- 1. oifcfg 주소와 GV$CLUSTER_INTERCONNECTS 주소가 다른 이유는?
- 2. 인터커넥트 진단 네 단계와 ping 의 한계는?
- 3. gc cr block 2-way 와 gc current block busy 의 해석은?
- 4. GV$SEGMENT_STATISTICS 를 봐야 하는 이유는?
- 5. group by thread# 점검의 결함과 해결은?
- 6. 두 번째 장애 때 CRS 가 재기동하지 않은 이유는?

실습 개요

- 01 인터커넥트와 HAIP 확인
- 02 ★ gc 경합 만들기 — 양 노드 40초 부하
- 03 ★ 경합 원인 객체 특정, 끝나면 ts_ch20 삭제
- 05 종합 점검 스크립트 작성과 결함 수정
- 자습: 04 TFA 로그 수집, 06 장애 시나리오
- 파일: RAC/20장/20장_실습_NN_*.txt